🚀 Catapult

Cahier des Charges — v1.0

Mise à jour automatique de catégorie Twitch

Spring Boot 4 · PostgreSQL · Thymeleaf · Docker

Mars 2026

Table des matières

1. Présentation du projet

2. Architecture globale

2.1 Stack technique

2.2 Composants principaux

2.3 Flux principal

2.4 Infrastructure Docker

3. Authentification & gestion des utilisateurs

4. Game Getters

5. Système de bindings

6. Intégration Twitch

7. Système d'événements Spring

8. Extensibilité — Commandes chat & features futures

9. Configuration

10. UI / Interface Thymeleaf

11. Persistance — Schéma PostgreSQL

Références & Documentation

Cahier des Charges — Catapult

1. Présentation du projet

1.1 Objectif

Catapult est une application web auto-hébergée qui automatise la mise à jour de la catégorie Twitch d'un streamer en fonction du jeu détecté sur son PC. Lorsqu'un changement de jeu est détecté, Catapult applique automatiquement la catégorie Twitch correspondante, ainsi que les Content Classification Labels (CCLs) associés, sans aucune intervention manuelle de la part du streamer.

1.2 Public cible

Catapult s'adresse aux streamers Twitch souhaitant éliminer la friction liée au changement manuel de catégorie en cours de stream. L'application est conçue pour être déployée par un administrateur technique (via Docker) et utilisée par plusieurs streamers sur une même instance.

1.3 Périmètre v1

Inclus :

- Détection du jeu en cours via Steam et Discord (OAuth2)
- Mise à jour automatique de la catégorie et des CCLs via l'API Twitch
- Gestion des bindings jeu ↔ catégorie Twitch, avec résolution automatique via IGDB
- Support multi-utilisateurs sur une instance unique, avec isolation complète des données
- Interface web Thymeleaf pour la configuration (connexions, bindings, settings)
- Commandes textuelles basiques dans le chat Twitch via EventSub
- Déploiement Docker avec `docker-compose.yml` fourni

Hors périmètre v1 :

- API publique exposée aux tiers
- Support mobile
- Architecture multi-instance ou haute disponibilité
- Notifications Discord, webhooks sortants, statistiques de jeu

1.4 Déploiement

Catapult est distribué en open-source. Son déploiement repose sur Docker Compose et ne requiert aucune dépendance externe au-delà d'une instance PostgreSQL et des credentials

OAuth2 des providers (Twitch, Steam, Discord). Une instance standard est opérationnelle en quelques minutes à partir du `docker-compose.yml` fourni dans le repository.

2. Architecture globale

2.1 Stack technique

Couche	Technologie
Framework	Spring Boot 4.0.3
Langage	Java (dernière LTS compatible)
Base de données	PostgreSQL
Templates	Thymeleaf
Sécurité	Spring Security + Spring OAuth2 Client
Build	Maven
Déploiement	Docker + Docker Compose

Une seule instance applicative supporte l'ensemble des utilisateurs. Aucune architecture multi-instance n'est prévue à ce stade.

2.2 Composants principaux

Composant	Rôle
`SchedulerService`	Scheduler global `@Scheduled`, itère sur les utilisateurs actifs et orchestre la chaîne de getters
`GameGetter`	Interface commune implémentée par `SteamGameGetter` et `DiscordGameGetter`
`GameStateService`	Maintient en mémoire le dernier jeu détecté par utilisateur pour la déduplication inter-cycles
`GameEventListener`	Listener Spring Events, résout le binding correspondant et délègue à `TwitchService`
`TwitchService`	Appels API Twitch (modification de chaîne, révocation de token)
`BindingService`	Résolution et gestion des bindings, orchestration de la recherche IGDB
`IgdbService`	Appels API IGDB, preload de la liste de jeux, gestion du cache TTL
`AccountService`	Cycle de vie des comptes : création, mise en `PENDING_DELETION`, recovery, suppression définitive
`TwitchChatListener`	Réception des messages Twitch via EventSub, publication des `ChatCommandEvent`
`ChatCommandListener`	Reçoit les `ChatCommandEvent` et délègue l'exécution au `CommandRegistry`
`CommandRegistry`	Registre centralisé des commandes chat disponibles et de leur logique

2.3 Flux principal

```
┌──────────────────────────────────────────────────────────────┐
│                        SchedulerService                      │
│                (@Scheduled, toutes les N secondes)           │
└──────────────────────────────┬───────────────────────────────┘
                               │ itère sur les users actifs (botEnabled)
                               ▼
┌──────────────────────────────────────────────────────────────┐
│                       GameGetter chain                       │
│        SteamGameGetter → DiscordGameGetter (par priorité)    │
└──────────────────────────────┬───────────────────────────────┘
                               │ Optional<DetectedGame>
                               ▼
┌──────────────────────────────────────────────────────────────┐
│                       GameStateService                       │
│         Comparaison avec le dernier état connu               │
│         → pas de changement : cycle terminé                  │
│         → changement : publication d'un événement Spring     │
└──────────────────────────────┬───────────────────────────────┘
                               │ GameDetectedEvent / NoGameDetectedEvent
                               ▼
┌──────────────────────────────────────────────────────────────┐
│                      GameEventListener                       │
│    Résolution du binding via BindingService (+ IGDB si absent)  │
│    Vérification statut (INCOMPLETE, ignored)                 │
└──────────────────────────────┬───────────────────────────────┘
                               │ Binding valide
                               ▼
┌──────────────────────────────────────────────────────────────┐
│                        TwitchService                         │
│           PATCH /helix/channels (game_id + CCLs)             │
└──────────────────────────────────────────────────────────────┘
```

2.4 Infrastructure Docker

Le repository fournit un `Dockerfile` et un `docker-compose.yml` prêts à l'emploi. Le `docker-compose.yml` définit deux services :

- `app` : image Spring Boot, expose le port HTTP, reçoit les secrets via variables d'environnement
- `db` : image PostgreSQL officielle, avec volume persistant pour les données

Aucune valeur sensible (secrets OAuth2, clé de chiffrement, mot de passe base de données) n'est embarquée dans l'image. Toutes ces valeurs sont injectées à l'exécution via des variables

d'environnement Docker. Les migrations du schéma PostgreSQL sont gérées automatiquement au démarrage de l'application via **Flyway**, intégré à Spring Boot — aucune initialisation manuelle de la base n'est requise.

3. Authentification & gestion des utilisateurs

3.1 Providers OAuth2

L'authentification repose sur Spring Security et Spring OAuth2 Client. Trois providers sont supportés :

- **Twitch** (provider principal) : la connexion Twitch crée le compte applicatif et sert d'identifiant primaire. La session Spring Security est basée sur le `twitchId` de l'utilisateur.
- **Steam** et **Discord** (providers secondaires) : connectés depuis les settings après la création du compte. Ils sont liés au `UserAccount` existant et ne peuvent pas créer de compte indépendant.

3.2 Isolation des données

Chaque `UserAccount` est intégralement isolé. Aucune donnée — bindings, settings, tokens, état du getter — n'est partagée entre utilisateurs. La seule configuration commune à l'ensemble de l'instance est celle définie dans le fichier `.properties` (client IDs, secrets, intervals, mapping CCL).

Les tokens OAuth2 (access token + refresh token) de chaque provider sont stockés en base, chiffrés. Spring OAuth2 Client gère leur rafraîchissement automatique avant chaque appel API.

3.3 Activation du bot

Le `UserAccount` expose un booléen `botEnabled` modifiable depuis la page settings sans impact sur les tokens ou les données. Lorsque `botEnabled` est `false`, le scheduler global ignore l'utilisateur et aucun événement Spring n'est publié pour son compte.

3.4 Déconnexion d'un provider secondaire

Si l'utilisateur déconnecte Steam ou Discord depuis ses settings, les tokens du provider concerné sont révoqués (dans la mesure du possible) et supprimés en base. Le getter correspondant est retiré de la chaîne de résolution. Le compte applicatif et les bindings existants restent intacts.

3.5 Suppression de compte

La suppression de compte suit un processus en deux temps pour permettre une période de récupération :

- **Initiation** : après confirmation explicite de l'utilisateur (re-saisie du nom d'utilisateur ou équivalent), le compte passe en statut `PENDING_DELETION` avec un timestamp. Le bot est immédiatement mis en pause et l'utilisateur voit un bandeau d'avertissement dans l'UI indiquant le temps restant avant suppression définitive.
- **Délai de récupération** : la durée est configurable via `app.account.deletion-delay-days` (défaut recommandé : 7 jours). Pendant ce délai, l'utilisateur peut se reconnecter avec son compte Twitch pour annuler la suppression et restaurer le compte à son état normal.
- **Suppression définitive** : un job `@Scheduled` s'exécute quotidiennement et traite les comptes `PENDING_DELETION` dont le délai est dépassé. Il effectue dans l'ordre : révocation du token Twitch (`POST /oauth2/revoke`), révocation du token Discord (`DELETE /oauth2/applications/@me/authorize`), suppression du token Steam en base (pas d'endpoint de révocation officiel), puis suppression en cascade de toutes les données de l'utilisateur. La session Spring Security est invalidée immédiatement.

4. Game Getters

4.1 Interface commune

Tous les game getters implémentent une interface commune `GameGetter` exposant une méthode `Optional getCurrentGame(UserAccount)` . Un `DetectedGame` contient le `sourceId` , le `sourceType` et le `sourceName` du jeu détecté. Cette abstraction permet d'ajouter de nouveaux getters à l'avenir sans modifier le moteur de polling.

4.2 Getters disponibles

SteamGameGetter : interroge l'API Steam via l'endpoint `GetRecentlyPlayedGames` (v0001). Le jeu considéré "en cours" est identifié via le champ `gameid` retourné par l'API, qui correspond à l' `appId` Steam du jeu actuellement joué. L'appel utilise le token Steam de l'utilisateur.

DiscordGameGetter : interroge l'endpoint `/users/@me` de l'API Discord avec le scope `activities.read` . Seules les activités de type `PLAYING` sont retenues, les autres (musique, IDE, etc.) sont ignorées.

4.3 Ordre de priorité et activation

Chaque utilisateur définit un ordre de priorité entre ses getters connectés. Cet ordre est stocké en base sous forme de liste ordonnée associée au compte. Un getter peut être désactivé individuellement sans révoquer sa connexion OAuth — il est alors simplement exclu de la chaîne de résolution.

Lors d'un cycle de polling, les getters sont appelés dans l'ordre de priorité de l'utilisateur. La chaîne s'arrête au premier getter retournant un résultat non vide. Si un getter échoue (erreur

réseau, token expiré, HTTP 429), l'erreur est loggée et le getter suivant est interrogé sans interrompre le cycle.

4.4 Scheduler global

Un scheduler global annoté `@Scheduled` tourne à intervalle fixe et itère sur l'ensemble des utilisateurs ayant le bot actif. L'intervalle est configurable via la propriété `app.polling.interval-seconds` (défaut : 60). Un événement Spring n'est publié que si le jeu détecté a changé par rapport au poll précédent, afin d'éviter des appels Twitch inutiles. L'état du dernier jeu détecté par utilisateur est conservé en mémoire.

4.5 Cas "aucun jeu détecté"

Si tous les getters actifs retournent vide sur un cycle donné, le système passe en état `NO_GAME`. Le comportement dans cet état est configurable par l'utilisateur :

- **Ne rien faire** : la catégorie Twitch reste inchangée
- **Appliquer une catégorie par défaut** : l'utilisateur définit une catégorie de fallback (champ optionnel dans les settings, sélectionné via la datalist IGDB)

Ce comportement est représenté par un `NoGameDetectedEvent` publié via `ApplicationEventPublisher`, traité par le même listener que les événements de détection.

4.6 Getters futurs (hors scope v1)

Les providers suivants sont identifiés comme candidats pour de futures versions. Chacun implémenterait l'interface `GameGetter` sans modification du core :

Provider	Faisabilité	Remarques
Xbox / Microsoft Game Pass	✅ Réaliste (v2)	API Xbox Live OAuth2 officielle, expose le jeu actif sur Windows via la Xbox app
Battle.net	✅ Réaliste (v2)	API Blizzard OAuth2 officielle, couvre les titres Blizzard uniquement
Ubisoft Connect	✅ Réaliste (v2)	API Ubisoft OAuth2 disponible, scope limité aux jeux Ubisoft
GOG Galaxy	⚠️ API locale	Pas d'API cloud publique — nécessite une intégration via le client local GOG Galaxy
EA App / Origin	⚠️ API locale	Pas d'API officielle publique — intégration fragile via le client local EA
Epic Games Store	⚠️ Pas d'API	Aucune API officielle de détection en temps réel — scraping local uniquement
Agent système local	🔧 Universel	Un agent léger sur le PC de l'utilisateur détecte les processus actifs et reporte à Catapult via HTTP/WebSocket — couvre tous les launchers sans API, mais implique un composant supplémentaire à déployer côté client

5. Système de bindings

5.1 Définition d'un binding

Un binding est l'association entre un jeu détecté par un game getter et une catégorie Twitch à appliquer. Chaque binding appartient à un utilisateur et est isolé des autres comptes. Sa structure est la suivante :

- `sourceId` : identifiant du jeu côté source (appId Steam, applicationId Discord, ou null si ajout manuel)
- `sourceType` : `STEAM`, `DISCORD`, ou `MANUAL`
- `sourceName` : nom du jeu tel que retourné par le game getter
- `twitchGameId` : identifiant IGDB/Twitch de la catégorie cible (nullable si `INCOMPLETE`)
- `twitchGameName` : nom lisible de la catégorie Twitch
- `ccls` : liste des `content_classification_labels` à appliquer (peut être vide)
- `status` : `AUTO` (créé par résolution IGDB), `MANUAL` (créé ou modifié par l'utilisateur), `INCOMPLETE` (IGDB n'a pas trouvé de correspondance)
- `ignored` : booléen — si `true`, aucune action Twitch n'est déclenchée pour ce jeu

5.2 Résolution automatique via IGDB

Lorsqu'un jeu est détecté sans binding existant, la résolution se déroule dans l'ordre suivant :

- **Recherche par `external_games`** : si le `sourceType` est `STEAM`, on interroge IGDB avec le champ `external_games` (category = 1, uid = appId Steam) pour obtenir un match direct et fiable.
- **Fallback textuel** : si aucun résultat n'est trouvé via l' `externalId`, une recherche par nom est effectuée sur l'endpoint `/games` d'IGDB. Le premier résultat est retenu.
- **Aucun résultat** : si IGDB ne retourne rien, un binding en statut `INCOMPLETE` est créé avec `twitchGameId` null. Ce binding apparaît dans l'UI et l'utilisateur doit le compléter manuellement.

Le binding résolu automatiquement est appliqué immédiatement, sans validation préalable de l'utilisateur. Il reste modifiable à tout moment.

5.3 Preload de la liste IGDB

Au démarrage de l'application, une liste de jeux est préchargée depuis IGDB et stockée en mémoire (ou cache local). Cette liste alimente un composant `` dans l'UI, permettant une saisie assistée lors de la création ou modification manuelle d'un binding. Un TTL est configurable dans le fichier `.properties` pour contrôler la fréquence de refresh de cette liste.

5.4 Jeux ignorés

Un utilisateur peut marquer un jeu comme ignoré directement depuis la liste des bindings. Lorsqu'un jeu ignoré est détecté, le moteur de polling ne déclenche aucun événement de changement de catégorie Twitch. Le binding reste présent dans la liste et le flag peut être retiré à tout moment.

5.5 Modification manuelle

Depuis l'UI, l'utilisateur peut pour chaque binding :

- Modifier la catégorie Twitch cible via la datalist IGDB
- Ajouter, modifier ou retirer des CCLs associés
- Basculer le flag `ignored`
- Supprimer le binding (il pourra être recréé automatiquement à la prochaine détection)

6. Intégration Twitch

6.1 Modification de la chaîne

Le `TwitchService` expose une méthode `updateChannel(UserAccount, Binding)` qui effectue un appel `PATCH /helix/channels` avec les champs `game_id` et `content_classification_labels`. Le scope OAuth2 requis est `channel:manage:broadcast`. Le token Twitch de l'utilisateur est récupéré depuis son `UserAccount` et rafraîchi automatiquement par Spring OAuth2 Client si expiré.

6.2 Gestion des CCLs

Les CCLs envoyés à Twitch sont déterminés via un système de mapping en deux étapes.

Étape 1 — Mapping IGDB → Twitch via `.properties` : lors de la résolution automatique d'un binding, `IgdbService` récupère les thèmes et descripteurs IGDB du jeu (champs `themes`, `keywords`, `age_ratings`). Le fichier `.properties` définit une table de traduction entre les identifiants IGDB et les valeurs CCL Twitch, sous la forme `app.mapping.ccl.=`. Par exemple :

```
app.mapping.ccl.violence=ViolentGraphic
app.mapping.ccl.mature=MatureGame
app.mapping.ccl.sexual_content=SexualThemes
app.mapping.ccl.drugs=DrugUse
app.mapping.ccl.gambling=Gambling
app.mapping.ccl.profanity=ProfanityVulgarity
app.mapping.ccl.language_barrier=LanguageBarrier
```

Chaque valeur IGDB présente dans les données du jeu est traduite vers le CCL Twitch correspondant. Si aucune correspondance n'existe dans le mapping, la valeur IGDB est ignorée. Les CCLs ainsi déduits sont stockés sur le binding et restent modifiables par l'utilisateur.

Étape 2 — Sélection par l'utilisateur : dans l'UI, l'utilisateur peut activer ou désactiver individuellement chaque CCL d'un binding via un multiselect. Les CCLs suggérés automatiquement sont pré-cochés, mais l'utilisateur a le dernier mot.

Les valeurs valides côté Twitch sont un enum fixe : `MatureGame`, `ViolentGraphic`, `SexualThemes`, `LanguageBarrier`, `DrugUse`, `Gambling`, `ProfanityVulgarity`. Si l'utilisateur a désactivé la feature CCL globalement dans ses settings, le champ `content_classification_labels` est intégralement omis de la requête PATCH, quelle que soit la configuration du binding ou du `.properties`.

6.3 Gestion des erreurs

- **HTTP 401** : le token est considéré révoqué ou invalide. Le bot est automatiquement mis en pause pour le compte concerné et une notification est affichée dans l'UI invitant l'utilisateur à reconnecter son compte Twitch.
- **HTTP 429** : la requête est retentée avec un délai exponentiel. Le scheduler global n'est pas bloqué pendant ce délai.
- **Autres erreurs** : loggées au niveau `ERROR` avec le contexte utilisateur, sans interruption du cycle de polling pour les autres utilisateurs.

6.4 Découplage

`TwitchService` ne connaît pas le système d'événements Spring. Il reçoit uniquement un `UserAccount` et un `Binding` en paramètres, ce qui le rend testable indépendamment et réutilisable par d'autres composants futurs. Chaque appel réussi est loggué au niveau `INFO` avec le `game_id` et les CCLs appliqués.

7. Système d'événements Spring

7.1 Événements

Deux événements couvrent l'ensemble des états possibles du moteur de polling :

- `GameDetectedEvent(UserAccount, DetectedGame)` : publié lorsqu'un jeu est détecté et que celui-ci diffère du dernier jeu connu pour cet utilisateur.
- `NoGameDetectedEvent(UserAccount)` : publié lorsque tous les getters actifs retournent vide et que l'utilisateur était précédemment en train de jouer.

Les deux classes étendent `ApplicationEvent` et sont publiées via `ApplicationEventPublisher`. Les événements sont synchrones par défaut (même thread que le scheduler global), ce qui simplifie la gestion des erreurs et évite des problèmes de concurrence non nécessaires à ce stade.

7.2 Listener principal

Un `GameEventListener` annoté `@EventListener` centralise le traitement des événements. Son flux de traitement pour un `GameDetectedEvent` est le suivant :

- Recherche du binding correspondant au `sourceId` + `sourceType` du `DetectedGame` dans les bindings de l'utilisateur.
- Si aucun binding n'existe, délégation au service de résolution IGDB (cf. section 5.2). Un nouveau binding est créé et l'événement est retraité immédiatement.
- Si le binding est en statut `INCOMPLETE` ou a le flag `ignored` à `true`, le listener s'arrête sans appel Twitch.

- Si le binding est valide, délégation au `TwitchService` pour le changement de catégorie et l'application des CCLs.

  Pour un `NoGameDetectedEvent`, le listener consulte la configuration de l'utilisateur : si une catégorie de fallback est définie, elle est appliquée via `TwitchService` ; sinon, aucune action n'est effectuée.

  Chaque événement reçu est loggué au niveau `DEBUG` avec l'identifiant utilisateur et le jeu détecté, pour faciliter le suivi en production.

7.3 État courant par utilisateur

Un `GameStateService` maintient en mémoire une `Map` représentant le dernier jeu connu pour chaque utilisateur actif. Le scheduler global consulte cet état avant de publier un événement : si le jeu détecté est identique au précédent, aucun événement n'est publié. Cet état est réinitialisé lorsque l'utilisateur désactive son bot.

7.4 Extensibilité

L'architecture événementielle permet d'ajouter de nouveaux listeners sans modifier le scheduler ni le listener principal. Un futur listener pour les commandes chat Twitch, les notifications Discord, ou tout autre feature, peut s'abonner aux mêmes `GameDetectedEvent` et `NoGameDetectedEvent` de façon indépendante.

8. Extensibilité — Commandes chat & features futures

8.1 Commandes textuelles Twitch (v1)

Les commandes chat sont une feature optionnelle, activable/désactivable globalement par l'utilisateur depuis ses settings. Elles reposent sur EventSub pour la réception des messages du chat Twitch.

Un `TwitchChatListener` distinct du `GameEventListener` traite les messages entrants. Lorsqu'une commande valide est reconnue, il publie un `ChatCommandEvent(UserAccount, command, args, senderRole)` via `ApplicationEventPublisher`. Un `ChatCommandListener` dédié reçoit cet événement et délègue l'exécution au `CommandRegistry`, qui centralise l'ensemble des commandes disponibles et leur logique associée. Cette séparation garantit que l'ajout de nouvelles commandes ne nécessite aucune modification du listener ni du core métier, et que d'autres composants peuvent s'abonner aux `ChatCommandEvent` de façon indépendante.

Chaque commande expose :

- Un nom (ex. `!game`)
- Une action associée

- Un niveau de permission requis : `BROADCASTER`, `MODERATOR`, ou `EVERYONE`

Les permissions sont configurables par l'utilisateur dans ses settings.

Commandes natives prévues en v1 :

Commande	Description	Permission par défaut
`!game`	Affiche dans le chat le jeu actuellement détecté	`EVERYONE`
`!setgame`	Force manuellement une catégorie Twitch pour la session en cours	`MODERATOR`

8.2 Architecture orientée extension

L'ensemble du système est conçu pour accueillir de nouvelles features sans réécriture du core :

- **Nouveaux game getters** : implémenter `GameGetter` et l'enregistrer dans la chaîne de résolution suffit.
- **Nouveaux listeners** : tout composant peut s'abonner aux `GameDetectedEvent` et `NoGameDetectedEvent` existants via `@EventListener` sans modifier le scheduler ni le listener principal.
- **Nouvelles commandes chat** : enregistrer une nouvelle entrée dans le `CommandRegistry`.

8.3 Features futures identifiées (hors scope v1)

Les éléments suivants sont documentés pour référence future et ne font pas partie du périmètre initial :

- Notifications Discord à chaque changement de catégorie Twitch
- Webhook sortant configurable (POST vers une URL définie par l'utilisateur à chaque événement)
- Statistiques de temps de jeu par utilisateur, consultables dans le dashboard
- Export / import des bindings en JSON
- Structure en modules Maven distincts (`core`, `twitch`, `discord`, `steam`) pour une meilleure séparation des responsabilités à plus grande échelle

9. Configuration

Toutes les propriétés applicatives sont définies dans `application.properties` (ou `application.yml`). Les valeurs sensibles (secrets, clé de chiffrement) doivent être injectées via variables d'environnement en production, notamment dans un contexte Docker.

9.1 Polling & scheduler

```
# Intervalle de polling en secondes (défaut : 60)
app.polling.interval-seconds=60

# TTL du refresh de la liste IGDB préchargée en heures (défaut : 24)
app.igdb.preload-ttl-hours=24
```

9.2 Gestion des comptes

```
# Délai en jours avant suppression définitive d'un compte PENDING_DELETION (défaut : 7)
app.account.deletion-delay-days=7
```

9.3 OAuth2 — Providers

```
# Twitch
spring.security.oauth2.client.registration.twitch.client-id=
spring.security.oauth2.client.registration.twitch.client-secret=

# Steam
spring.security.oauth2.client.registration.steam.client-id=
spring.security.oauth2.client.registration.steam.client-secret=

# Discord
spring.security.oauth2.client.registration.discord.client-id=
spring.security.oauth2.client.registration.discord.client-secret=
```

9.4 IGDB

```
# Client ID IGDB (correspond au client Twitch — Twitch et IGDB partagent la même authentif
app.igdb.client-id=
```

9.5 Mapping CCL (IGDB → Twitch)

Chaque propriété associe une valeur IGDB (thème, descripteur) à un label CCL Twitch. Ce mapping est défini par l'administrateur de l'instance et s'applique à tous les utilisateurs comme base de suggestion automatique lors de la résolution d'un binding via IGDB. La clé est le terme IGDB, la valeur est l'un des sept CCLs valides côté Twitch.

```
# Format : app.mapping.ccl.<valeur_igdb>=<TwitchCCL>
app.mapping.ccl.violence=ViolentGraphic
app.mapping.ccl.mature=MatureGame
app.mapping.ccl.sexual_content=SexualThemes
app.mapping.ccl.drugs=DrugUse
app.mapping.ccl.gambling=Gambling
app.mapping.ccl.profanity=ProfanityVulgarity
app.mapping.ccl.language_barrier=LanguageBarrier
```

9.6 Base de données

```
spring.datasource.url=jdbc:postgresql://localhost:5432/streamlink
spring.datasource.username=
spring.datasource.password=
```

9.7 Sécurité

```
# Clé de chiffrement AES utilisée pour les tokens OAuth2 stockés en base
app.security.token-encryption-key=
```

10. UI / Interface Thymeleaf

10.1 Pages

L'application expose quatre pages principales :

- `/login` : page de connexion avec un unique bouton "Se connecter avec Twitch". Aucune inscription manuelle n'est possible.
- `/dashboard` : page d'accueil après connexion, affichant le jeu en cours détecté, le dernier changement de catégorie Twitch effectué, et l'état du bot.
- `/settings` : page unique de configuration utilisateur, organisée en sections verticales.
- `/bindings` : liste complète des bindings de l'utilisateur avec filtres, recherche et actions rapides.

Un bandeau d'avertissement global est affiché sur toutes les pages lorsque le compte est en statut `PENDING_DELETION`. Ce bandeau indique la date de suppression définitive, un countdown, et un bouton "Annuler la suppression".

10.2 Dashboard

Le dashboard affiche :

- L'état du bot (actif / inactif) avec un indicateur visuel clair
- Le jeu actuellement détecté par le getter actif (nom, source, icône du provider)
- La dernière action Twitch effectuée (catégorie appliquée, timestamp)

10.3 Page Settings

Connexions

Trois cartes, une par provider (Twitch, Steam, Discord), affichant l'état de connexion et les actions disponibles. Twitch ne peut pas être déconnecté indépendamment (cela équivaudrait à une déconnexion de session) : seuls Steam et Discord disposent d'un bouton "Déconnecter".

Bot & Game Getters

- Toggle principal `botEnabled` avec libellé explicite ("Activer / Désactiver le bot").
- Liste ordonnée des getters connectés et activés, réorganisable via des boutons ↑ / ↓. Seuls les providers effectivement connectés apparaissent dans cette liste.
- Toggle individuel par getter pour l'activer ou le désactiver sans le déconnecter.

Catégorie par défaut (aucun jeu détecté)

Champ texte associé à un alimenté par la liste IGDB préchargée, permettant de définir la catégorie Twitch de fallback. Laisser le champ vide équivaut à l'option "Ne rien faire" (valeur `null` en base).

Content Classification Labels (CCL)

- Toggle global "Activer la feature CCL" pour le compte. Si désactivé, aucun CCL n'est envoyé à Twitch, quelle que soit la configuration des bindings.
- Indication visuelle non modifiable du mapping global défini par l'administrateur de l'instance dans le `.properties`.

Suppression de compte

Zone visuellement distincte (couleur danger). Un bouton "Supprimer mon compte" ouvre une modale de confirmation requérant la re-saisie du nom d'utilisateur Twitch. Si le compte est déjà en statut `PENDING_DELETION`, cette zone affiche à la place la date de suppression définitive et un bouton "Annuler la suppression".

10.4 Page Bindings

Un tableau paginé liste l'ensemble des bindings de l'utilisateur avec les colonnes suivantes : source (icône du provider + nom du jeu), catégorie Twitch cible, CCLs associés, statut (`AUTO` , `MANUAL` , `INCOMPLETE`), flag ignoré, et actions disponibles.

Les bindings en statut `INCOMPLETE` sont signalés par un badge visuel distinctif invitant l'utilisateur à les compléter.

Filtres disponibles : par statut (`AUTO` , `MANUAL` , `INCOMPLETE`) et par type de source (`STEAM` , `DISCORD` , `MANUAL`).

Modale d'édition : ouverte au clic sur un binding, elle expose :

- Un champ texte avec IGDB pour modifier la catégorie Twitch cible
- Un multiselect des CCLs disponibles (valeurs fixes Twitch)
- Un toggle "Ignorer ce jeu"

11. Persistance — Schéma PostgreSQL

11.1 Table `user_account`

Représente un compte applicatif. L'identifiant primaire applicatif est un UUID interne ; `twitch_id` est l'identifiant unique externe.

Colonne	Type	Contraintes
`id`	UUID	PK
`twitch_id`	varchar	unique, not null
`twitch_username`	varchar	not null
`bot_enabled`	boolean	not null, défaut `true`
`status`	enum (`ACTIVE` , `PENDING_DELETION`)	not null, défaut `ACTIVE`
`deletion_requested_at`	timestamp	nullable
`created_at`	timestamp	not null
`updated_at`	timestamp	not null

11.2 Table `oauth_token`

Stocke les tokens OAuth2 par provider pour chaque utilisateur. Les valeurs `access_token` et `refresh_token` sont chiffrées en base via la clé `app.security.token-encryption-key` .

Colonne	Type	Contraintes
`id`	UUID	PK
`user_id`	UUID	FK → `user_account`, not null
`provider`	enum (`TWITCH` , `STEAM` , `DISCORD`)	not null
`access_token`	text	not null, chiffré
`refresh_token`	text	nullable, chiffré
`expires_at`	timestamp	nullable

Contrainte unique sur `(user_id, provider)`.

11.3 Table `getter_config`

Définit l'ordre de priorité et l'état d'activation de chaque game getter par utilisateur.

Colonne	Type	Contraintes
`id`	UUID	PK
`user_id`	UUID	FK → `user_account`, not null
`provider`	enum (`STEAM` , `DISCORD`)	not null
`priority`	int	not null
`enabled`	boolean	not null, défaut `true`

Contrainte unique sur `(user_id, provider)`.

11.4 Table `game_binding`

Représente l'association entre un jeu détecté et une catégorie Twitch pour un utilisateur donné.

Colonne	Type	Contraintes
`id`	UUID	PK
`user_id`	UUID	FK → `user_account`, not null
`source_id`	varchar	nullable
`source_type`	enum (`STEAM` , `DISCORD` , `MANUAL`)	not null
`source_name`	varchar	not null
`twitch_game_id`	varchar	nullable
`twitch_game_name`	varchar	nullable
`status`	enum (`AUTO` , `MANUAL` , `INCOMPLETE`)	not null
`ignored`	boolean	not null, défaut `false`
`created_at`	timestamp	not null
`updated_at`	timestamp	not null

11.5 Table `binding_ccl`

Table de jointure stockant la liste des CCLs associés à un binding.

Colonne	Type	Contraintes
`binding_id`	UUID	FK → `game_binding` , not null
`ccl`	enum Twitch CCL	not null

Clé primaire composite sur `(binding_id, ccl)` .

11.6 Table `user_settings`

Stocke les préférences globales de l'utilisateur. Relation 1-1 avec `user_account` .

Colonne	Type	Contraintes
`user_id`	UUID	PK, FK → `user_account`
`ccl_feature_enabled`	boolean	not null, défaut `true`
`no_game_twitch_game_id`	varchar	nullable
`no_game_twitch_game_name`	varchar	nullable

11.7 Index

```
-- Résolution rapide du binding au moment du poll
CREATE INDEX idx_binding_user_source ON game_binding(user_id, source_id, source_type);

-- Job de suppression quotidien des comptes PENDING_DELETION
CREATE INDEX idx_account_deletion ON user_account(status, deletion_requested_at);
```

Références & Documentation

Twitch

- API Twitch — Modify Channel Information (PATCH /helix/channels)
- Twitch EventSub — Documentation officielle
- Twitch — Révocation de tokens OAuth2
- Twitch — Scopes OAuth2 (channel:manage:broadcast)

Steam

- Steam Web API — GetRecentlyPlayedGames (v0001)
- Steam — Portail développeur & clés API

Discord

- Discord — OAuth2 Documentation
- Discord — GET /users/@me (activities.read)

IGDB

- IGDB API — Documentation des endpoints
- IGDB — External Games (mapping Steam appId)
- IGDB — Age Ratings (source CCL suggestions)

Providers futurs

- Xbox Live / GDK — Game Runtime API
- Battle.net — API OAuth2 Developer Guide
- Ubisoft Connect — Portail développeur

Stack technique

- Spring Boot 4 — Documentation officielle
- Spring Security — OAuth2 Client
- Thymeleaf — Documentation
- PostgreSQL — Documentation officielle
- Docker Compose — Documentation officielle